Falsifier-Based Robustification for OpenSCvx

October 2026

1 Introduction

Robust trajectory optimization is vital for autonomous robotic systems operating under uncertain dynamics and environmental disturbances. While successive convexification (SCvx) enables nonlinear trajectory optimization, nominal solutions may remain vulnerable to disturbances. We investigate a counterexample-guided robustification framework for OpenSCvx that integrates adversarial falsification with trajectory optimization. A falsifier identifies disturbances causing continuous-time safety violations, and those disturbances are explicitly converted into constraints for subsequent SCvx iterations. The initial experiment tests whether this feedback improves sampled obstacle clearance. Comparisons of computational cost against existing optimization methods and of explicit constraint updates against neural policy fine-tuning require separate experiments.

2 State-of-the-Art

Mutapcic and Boyd [1] developed an optimization method that repeatedly searches for worst-case scenarios and adds new constraints to prevent similar failures. Dawson and Fan [2] applied a related approach to trajectory planning where an optimizer learns from failures identified by a falsifier. Their method produced more robust trajectories while taking less than half the time of competing methods in their experiments.

Shi et al. [3] used RL (PPO) to discover disturbances that could cause quadrupedal robots to fail. They then retrained the robot's neural controllers using these failures to improve their robustness.

Abdul et al. [4] developed a SCvx method that plans trajectories while accounting for disturbances within a predefined bounded set. Their approach also accounts for errors caused by approximating nonlinear dynamics. Wang et al. [6] later accelerated this method using GPUs. Elango et al. [5] developed a way for SCvx to enforce safety constraints throughout the entire trajectory rather than only at selected time steps.

Building on these works, we add a falsifier to OpenSCvx that actively searches for disturbances that cause trajectory failures. Each discovered failure introduces a new scenario safety constraint and allows SCvx to replan. We investigate the resulting sampled clearance and computational cost.

3 Limitations

Existing robust SCvx methods [4, 6] account for bounded disturbances, but solving these optimization problems can be computationally heavy. Counterexample-guided methods [2] improve planning by learning from failures. Meanwhile, adversarial RL [3] improves neural controllers through fine-tuning rather than explicit constraint updates. This experiment examines adaptive falsification with continuous-time SCvx; it does not establish that this combination is novel.

4 Ideas

A unicycle model corresponds to a rover navigating a surface environment, and this has a wide range of applications, including, but not limited to, the following case studies:

- Surface exploration of Moon/Mars (the model corresponds to the physical system)
- Surface exploration of unknown Earth environments (the model simplifies a legged robot, but may be representative for planning)
- Planning of autonomous vehicles in traffic: e.g. car, trucks... (the model simplifies a car, but may be representative for planning)

5 Mathematical Formulation

We consider a planar unicycle with continuously actuated forward and angular velocities. The robot moves without lateral slip, and its dynamics are deterministic. The planning horizon $\mathcal{H} = [0, T]$ is fixed. Initial and terminal configurations are prescribed.

The state and control vectors are

$$\boldsymbol{x} = \begin{bmatrix} p_x \\ p_y \\ \theta \end{bmatrix}, \qquad \boldsymbol{u} = \begin{bmatrix} v \\ \omega \end{bmatrix}. \tag{1}$$

The nonlinear dynamics are

$$\boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u}) = \begin{bmatrix} v\cos\theta \\ v\sin\theta \\ \omega \end{bmatrix}. \tag{2}$$

Let $\boldsymbol{g}(\boldsymbol{x}) \leq \boldsymbol{0}$ denote the collision-avoidance constraints and $\boldsymbol{G}(\boldsymbol{u}) \leq \boldsymbol{0}$ the actuator limits. The objective is to minimize the traveled distance.

Problem 1 (P1)

$$\begin{aligned} \underset{\boldsymbol{x}(\cdot),\, \boldsymbol{u}(\cdot)}{\text{minimize}} \quad & \int_0^T v(t)\, dt \\ \text{subject to} \quad & \left| \begin{aligned} \dot{\boldsymbol{x}} &= \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u}) \\ \boldsymbol{g}(\boldsymbol{x}) &\leq \boldsymbol{0} \\ \boldsymbol{G}(\boldsymbol{u}) &\leq \boldsymbol{0} \\ \boldsymbol{x}(0) &= \boldsymbol{x}_{\text{init}} \\ \boldsymbol{x}(T) &= \boldsymbol{x}_{\text{goal}} \end{aligned} \right. \end{aligned} \tag{3}$$

The actuator limits are defined by

$$\boldsymbol{G}(\boldsymbol{u}) = \begin{bmatrix} -v \\ v - v_{\max} \\ \omega - \omega_{\max} \\ -\omega - \omega_{\max} \end{bmatrix}. \tag{4}$$

5.1 Counterexample-Guided Robustification

We extend Problem 1 with a bounded constant steering disturbance $b \in \mathcal{B} = [-\bar{b}, \bar{b}]$:

$$\dot{\boldsymbol{x}}_b = \begin{bmatrix} v\cos\theta_b \\ v\sin\theta_b \\ \omega + b \end{bmatrix}. \tag{5}$$

Adversarial Falsification. For a circular obstacle with center $\boldsymbol{c}$ and radius R, define the violation function

$$q(\boldsymbol{x}_b) = R^2 - \|\boldsymbol{p}_b - \boldsymbol{c}\|_2^2. \tag{6}$$

Given the current control $\boldsymbol{u}^j$, the falsifier searches for the worst-case disturbance and time:

$$\boxed{(b_j^*, t_j^*) \in \underset{b\in\mathcal{B},\ t\in[0,T]}{\operatorname{argmax}}\ q(\boldsymbol{x}_b(t; \boldsymbol{u}^j)).} \tag{7}$$

A positive maximum indicates a safety violation.

Counterexample Feedback. Starting with $\mathcal{S}_0 = \{0\}$, each discovered violating disturbance is retained:

$$\boxed{\mathcal{S}_{j+1} = \mathcal{S}_j \cup \{b_j^*\}.} \tag{8}$$

The next optimization introduces one state trajectory per retained disturbance, with shared controls:

$$\begin{aligned} \min_{\boldsymbol{u},\{\boldsymbol{x}_b\}} \quad & \int_0^T v(t)\,dt \\ \text{s.t.} \quad & \dot{\boldsymbol{x}}_b = \boldsymbol{f}_b(\boldsymbol{x}_b, \boldsymbol{u}), && b \in \mathcal{S}_{j+1}, \\ & q(\boldsymbol{x}_b(t)) \le 0, && \forall t,\ b \in \mathcal{S}_{j+1}, \\ & \boldsymbol{G}(\boldsymbol{u}(t)) \le \boldsymbol{0}, && \forall t \in [0,T], \\ & \boldsymbol{x}_b(0) = \boldsymbol{x}_{\text{init}}, && b \in \mathcal{S}_{j+1}, \\ & \boldsymbol{x}_0(T) = \boldsymbol{x}_{\text{goal}}. \end{aligned} \tag{9}$$

Integration into SCvx. For each scenario, SCvx linearizes the disturbed dynamics around a reference trajectory and applies the affine obstacle-avoidance restriction:

$$\boxed{q(\bar{\boldsymbol{p}}_b) - 2(\bar{\boldsymbol{p}}_b - \boldsymbol{c})^\top (\boldsymbol{p}_b - \bar{\boldsymbol{p}}_b) \le 0.} \tag{10}$$

This is a conservative convex restriction since $q(\boldsymbol{p}_b)$ equals the affine expression minus $\|\boldsymbol{p}_b - \bar{\boldsymbol{p}}_b\|_2^2$.

Continuous-time safety constraints can be handled using OpenSCvx's existing framework.

The outer loop alternates falsification and SCvx re-optimization, adding scenarios only when a violation is discovered.

Scope. Only nominal terminal-state feasibility is imposed. Safety over the full uncertainty set requires certified falsification or additional verification. Convergence and computational advantages of the combined algorithm remain to be established.

6 Development and Problem Solution

OpenSCvx solves nonlinear optimal control problems by iteratively writing and solving convex subproblems around a nominal trajectory.

1. A planar unicycle moves over a fixed 8 s horizon.
2. The vehicle starts at $(0, 0, 0)$ and must reach $(4, 0, 0)$.
3. The controls satisfy $0 \leq v \leq 1\,\mathrm{m/s}$ and $|\omega| \leq 0.8\,\mathrm{rad/s}$.
4. A circular obstacle centered at $(2, 0)$ has radius $0.55\,\mathrm{m}$.
5. The path must satisfy $\|\boldsymbol{p}(t) - (2, 0)^\top\|_2 \geq 0.55\,\mathrm{m}$ for all $t \in [0, 8]$.

A plot of the baseline trajectory and its worst disturbed rollout follows.

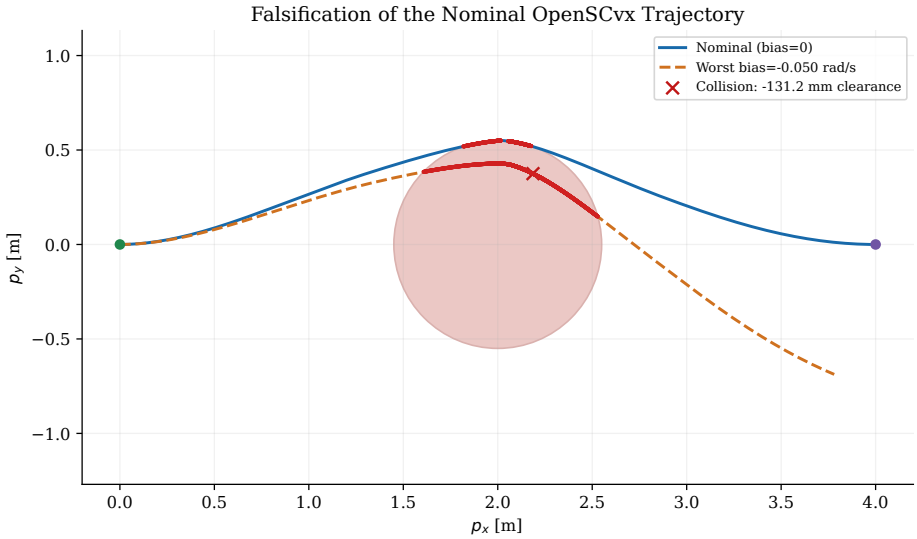


Figure 1: Baseline nominal trajectory and worst disturbed rollout. Red segments indicate independently simulated obstacle penetration.

7 Initial Experimental Results

The recorded experiment uses OpenSCvx 0.5.2 and the original $N = 5$ settings. A constant steering bias is searched over $[-0.05, 0.05]$ rad/s using 101 values. Final validation uses 501 bias values and 8001 time samples per rollout, with independent SciPy integration. The retained scenario set after one re-optimization is $\{0, -0.05\}$ rad/s.

Metric	Before	After
Worst sampled clearance (m)	−0.131209	−0.015776
Geometric path length (m)	4.181564	4.257311
Signed distance objective (m)	4.170101	4.240522
Maximum applied speed (m/s)	1.065051	1.015535
Minimum applied speed (m/s)	−0.088544	−0.108607

Both SCvx solves reported convergence. The final nominal trajectory has sampled clearance 0.083289 m, but the retained disturbed scenario still collides. The run therefore demonstrates scenario feedback and improved clearance, not a successful robust repair. Total runtime was approximately 73 s, including approximately 37 s for independent falsification and validation and 2 s in the optimizer loops. Remaining time includes initialization, compilation, post-processing, and figure generation.

Inspection of the installed interpolation found $\beta = N(\tau - \tau_k)$ over a segment of width $1/(N-1)$. Thus β reaches 1.25 for $N = 5$, and the applied controls extrapolate beyond the next knot. This causes between-node speed-bound violations; the dense `u_full` samples use a different interpolation. Independent integration of the reconstructed applied controls reproduces OpenSCvx's propagated pose within approximately 10^{-5}. Geometric length uses $\int |v|\, dt$, while the original objective uses $\int v\, dt$; these differ because applied speed becomes negative. These implementation and numerical limitations must be resolved before claiming physical feasibility or robustness. The baseline also has a small zero-bias collision. Finite-grid validation is not a continuous-uncertainty safety certificate.

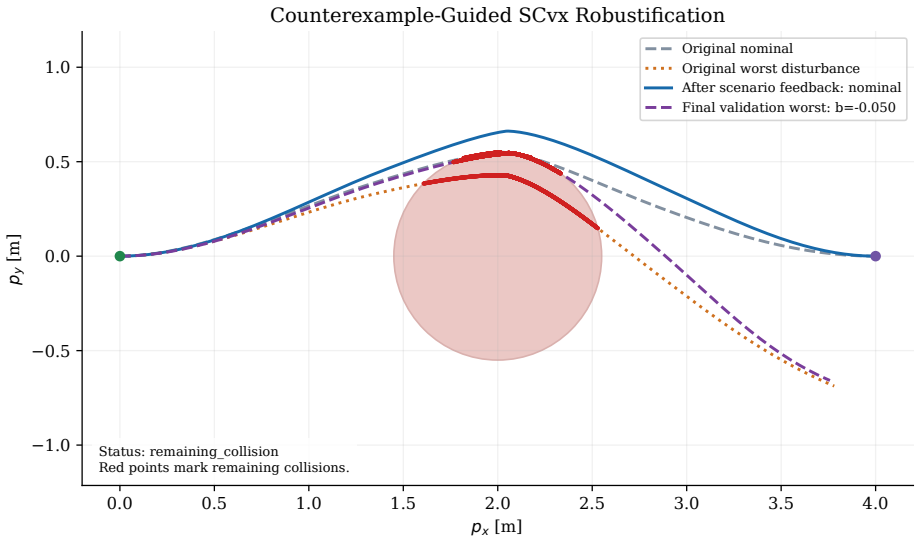


Figure 2: Before and after scenario feedback. Remaining collisions are explicitly marked.

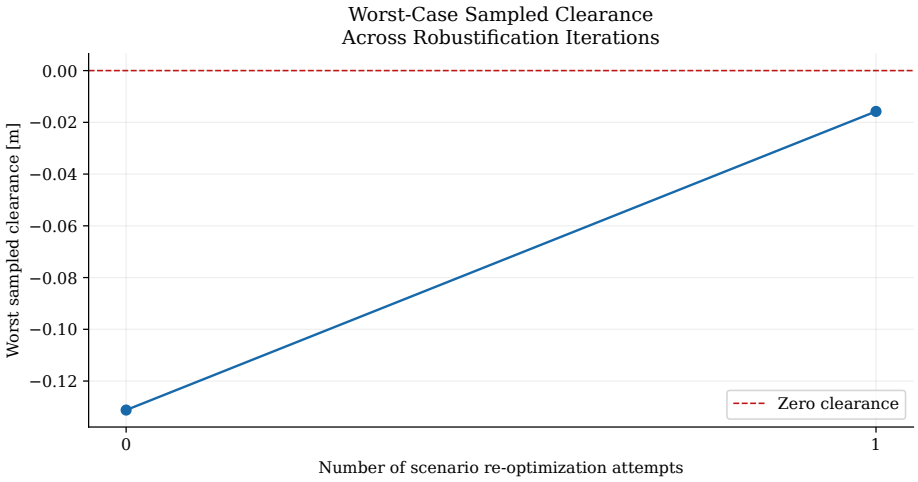


Figure 3: Worst sampled clearance over the same disturbance interval. Both measured values remain below zero.

8 Conclusions

One complete falsification, counterexample feedback, and OpenSCvx re-solve cycle improved sampled obstacle clearance but retained a collision and applied control-bound violations. The experiment does not establish robust feasibility, algorithmic novelty, or comparative computational advantages. The next step is to resolve the diagnosed interpolation and constraint accuracy issues before broader robustness experiments.

References

[1] A. Mutapcic and S. Boyd, "Cutting-set methods for robust convex optimization with pessimizing oracles," *Optimization Methods and Software*, vol. 24, no. 3, pp. 381–406, 2009. https://doi.org/10.1080/10556780802712889

[2] C. Dawson and C. Fan, "Robust Counterexample-guided Optimization for Planning from Differentiable Temporal Logic," arXiv:2203.02038, 2022. https://arxiv.org/abs/2203.02038

[3] F. Shi et al., "Rethinking Robustness Assessment: Adversarial Attacks on Learning-based Quadrupedal Locomotion Controllers," *Robotics: Science and Systems*, 2024. https://doi.org/10.15607/RSS.2024.XX.057

[4] A. T. Abdul, A. D. Saravanos, and E. A. Theodorou, "Nonlinear Robust Optimization for Planning and Control," *IEEE Conference on Decision and Control (CDC)*, 2025. https://doi.org/10.1109/CDC57313.2025.11312909

[5] P. Elango et al., "Continuous-time successive convexification for constrained trajectory optimization," *Automatica*, vol. 180, article 112464, 2025. https://doi.org/10.1016/j.automatica.2025.112464

[6] J. Wang, A. T. Abdul, and E. A. Theodorou, "cuNRTO: GPU-Accelerated Nonlinear Robust Trajectory Optimization," *Robotics: Science and Systems*, 2026. https://arxiv.org/abs/2603.02642